

Rapport TP1 : Reconnaissance de formes simples

Date : 15 janvier 2026

Anas DAGGAG - Jeremy DOUBLET - APP5 EIE

Introduction

Objectif du TP : Implémenter des méthodes de reconnaissance de formes géométriques simples (cercle, triangle, octogone, carré, rectangle) en utilisant différents descripteurs de forme.

Données :

- Images 00 à 04 : 5 exemples de référence (un par catégorie)
 - Images 05 à 26 : 22 images de test à classifier
 - Catégories : 0=cercle, 1=triangle, 2=octogone, 3=carré, 4=rectangle
-

Question 1 : Fonction `myShapeCompute` - Calcul des paramètres de forme

1.1. Calcul de l'aire

L'aire correspond au nombre de pixels de l'objet, disponible directement dans `stats[4]` retourné par `cv2.connectedComponentsWithStats`.

```
aire = stats[4] # Nombre de pixels à 1 dans l'objet
```

1.2. Calcul du périmètre

Le périmètre est calculé à partir du code de Freeman du contour.

a) Code de Freeman (`myFreemanCode`)

Encode chaque déplacement entre deux points consécutifs du contour selon 8 directions :

```
  3   2   1
   \  |  /
  4 - . - 0
   /  |  \
  5   6   7
```

Implémentation :

```
def mySignature(objet):
    contour, _ = cv2.findContours(objet, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
```

```

cnt=contour[0]
ch=np.mean(cnt[:, 0, 0]) # Centre de gravité x
cw=np.mean(cnt[:, 0, 1]) # Centre de gravité y
signature=[]

for point in cnt:
    x = point[0, 0]
    y = point[0, 1]
    distance = np.sqrt((x - ch)**2 + (y - cw)**2)
    signature.append(distance)

return(signature)

```

myFreemanCode(objet)

Calcule le code de Freeman pour encoder le contour (8 directions possibles).

```

def myFreemanCode(objet):
    contour,_ = cv2.findContours(objet,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_NONE)
    cnt=contour[0]
    p0 = cnt[0]
    code=[]

    for p in range(1, len(cnt)):
        p1 = cnt[p]
        dx = p1[0, 0] - p0[0, 0]
        dy = p1[0, 1] - p0[0, 1]
        dx = np.clip(dx, -1, 1)
        dy = np.clip(dy, -1, 1)
        code.append(Freeman[dy+1, dx+1])
        p0 = p1

    return(code)

```

myPerimeter(freeman_code)

Calcule le périmètre à partir du code de Freeman :

- Directions paires (0,2,4,6) : distance = 1
- Directions impaires (1,3,5,7) : distance = $\sqrt{2}$

```

def myPerimeter(freeman_code):
    perim=0.
    for p in range(0, len(freeman_code)):
        if freeman_code[p] % 2 == 0:
            perim += 1.0
        else:
            perim += np.sqrt(2)
    return(perim)

```

myShapeCompute(objet, stats)

Calcule tous les paramètres de forme :

- **Aire** : `stats[4]` (nombre de pixels à 1)
- **Périmètre** : calculé via le code de Freeman
- **Compacité** : $\text{périmètre}^2 / \text{aire}$ (minimale pour un cercle = $4\pi \approx 12.57$)
- **Pseudo-rectangularité** : $\text{aire} / (\text{largeur} \times \text{hauteur})$
- **Signature** : distance centre→contour
- **Moments de Hu** : 7 invariants de forme

3. Méthodes de reconnaissance implémentées

3.1. Reconnaissance par Compacité (`myCompacityAnalysis`)

Principe : Compare la compacité de chaque forme test avec celle des 5 exemples. La compacité est définie par :

$$\text{Compacité} = \frac{P^2}{A}$$

où P = périmètre et A = aire.

Algorithme :

1. Pour chaque objet à tester, calculer sa compacité
2. Trouver l'exemple ayant la compacité la plus proche (distance minimale)
3. Assigner la catégorie de cet exemple

Résultats :

```

prediction      : [2 0 2 4 1 1 3 1 0 3 3 3 3 1 4 1 1 1 2 3 0 0]
ground truth    : [2 2 2 4 4 4 4 4 0 3 3 3 3 1 1 1 1 1 2 5 0 0]
Accuracy        : 0.682 (68.2%)

```

Valeurs typiques de compacité observées :

- Cercles : ~13.4 - 13.9
- Octogones : ~13.2 - 13.7
- Carrés : ~15.3 - 18.3
- Triangles : ~23.3 - 23.6
- Rectangles : ~22.1 - 24.5

3.2. Reconnaissance par Moments de Hu (`myHuMomentsAnalysis`)

Principe : Utilise les 7 moments invariants de Hu, qui sont invariants par translation, rotation et changement d'échelle.

Algorithme :

1. Pour chaque objet, calculer les 7 moments de Hu
2. Calculer la distance euclidienne entre le vecteur de moments de l'objet test et ceux des exemples
3. Assigner la catégorie de l'exemple le plus proche

Résultats :

```
prediction      : [2 1 0 1 3 2 0 3 1 3 4 2 0 0 1 2 1 1 0 3 3 2]
ground truth    : [2 2 2 4 4 4 4 4 0 3 3 3 3 1 1 1 1 1 2 5 0 0]
Accuracy        : 0.227 (22.7%)
```

3.3. Reconnaissance par Analyse de Signature (*mySignatureAnalysis*)

Principe : Analyse les maxima locaux de la signature après filtrage gaussien. Le nombre de pics correspond au nombre de "coins" de la forme.

Algorithme :

1. Normaliser la signature (rotation du minimum au début)
2. Appliquer un filtre gaussien ($\sigma = \text{longueur_contour}/128$)
3. Détecter les maxima locaux avec `argrextrema(signature, np.greater)`
4. Classifier selon le nombre de maxima :
 - ≤ 2 maxima → **Cercle**
 - 3 maxima → **Triangle**
 - 4 maxima → **Carré ou Rectangle** (selon variance des distances)
 - 7-9 maxima → **Octogone**
 - Autre → **Autre**

Résultats :

```
prediction      : [2 2 2 3 3 3 3 3 5 3 3 3 3 1 1 1 1 1 2 5 5 3]
ground truth    : [2 2 2 4 4 4 4 4 0 3 3 3 3 1 1 1 1 1 2 5 0 0]
Accuracy        : 0.636 (63.6%)
```